

NEXUS

Needs Exchange & Unified Coordination System

Smart Resource Allocation & Data-Driven Volunteer Coordination

for Social Impact

System Architecture

Module Specifications

Data Models

Implementation Plan

TABLE OF CONTENTS

01	Executive Overview & Problem Diagnosis — 6 core failure domains
02	System Architecture — NEXUS Master Design — 6-layer structure
03	Layer 1 — Data Ingestion — Multimodal input channels
04	Layer 2 — Document Processing Engine — OCR · NLP · Geocoding · Dedup
05	Layer 3 — AI Intelligence Core — Urgency scoring · Heatmaps
06	Layer 4 — Coordination Engine — Matching · Dispatch · Tracking
07	Layer 5 — Presentation Layer — Dashboard · Apps · Portal
08	Layer 6 — Cross-Cutting Concerns — Security · Audit · Multi-tenancy
09	Module 01 — Household & Beneficiary Registry — The identity foundation
10	Module 02 — Ingestion Gateway
11	Module 03 — Document Processing Service
12	Module 04 — NLP Intelligence Service
13	Module 05 — Urgency Scoring Engine
14	Module 06 — Volunteer Registry Service
15	Module 07 — Matching Engine
16	Module 08 — Task Orchestration Service
17	Module 09 — Notification Service
18	Module 10 — Analytics & Reporting Service
19	Module 11 — Multi-Tenancy & Admin Service
20	Complete Data Models — All canonical schemas

21	Technology Stack — Justified tool selections
22	API Surface Specification — REST · GraphQL · WebSocket
23	Phased Implementation Roadmap — 44-week delivery plan
24	Non-Obvious Design Decisions — Critical engineering choices
25	Operational Considerations — DevOps · Monitoring · SLA
26	Compliance & Privacy — DPDP Act 2023 · Consent architecture

The brief states that community data is 'scattered across different places.' This is the symptom. The actual disease is a multi-layered coordination collapse across six distinct failure domains. No single-point intervention resolves this. Only a system designed to address all six simultaneously produces measurable change.

FD-1 — Data Origin & Semantic Gap

Field workers collect needs via paper surveys in rural areas, often in local dialects. Papers sit in offices for days. When digitized, context is lost. A note saying 'family near the temple has no food' becomes 'food shortage, unspecified household.'

Simultaneously, WhatsApp groups — the de facto communication layer for Indian NGOs — generate hundreds of unstructured, ephemeral reports daily that are invisible to any management system.

FD-2 — Invisible Household Duplication

Three NGOs may operate in the same slum, each running independent surveys, each building independent 'priority family' lists. The same household gets visited three times with duplicate supplies, while a worse-off neighboring household gets zero visits. Without a shared household registry, this duplication is structurally invisible and cannot be corrected.

FD-3 — Prioritization Vacuum

Even when data is collected, no scientifically defensible method exists to answer: 'Among 2,000 reported needs across 50 neighborhoods, which 10 do we address today with 15 volunteers?' Most coordinators use instinct, relationship, or recency. Resources flow to vocal communities rather than vulnerable ones.

FD-4 — Volunteer Mismatch

Coordinators maintain mental models or spreadsheets of volunteer availability. Task assignment takes 30–90 minutes per task. Matching uses only two variables: availability and geography. Missing: skill fit, language, cultural context, prior community relationship, and burnout risk tracking.

FD-5 — Invisible Outcomes

Once a volunteer is dispatched, the system loses visibility. Did the intervention succeed? Was the need fully met? Did new needs emerge? Without structured outcome capture, every dispatch is fire-and-forget. Organizations cannot learn. Funders cannot see impact. Chronic households remain invisible.

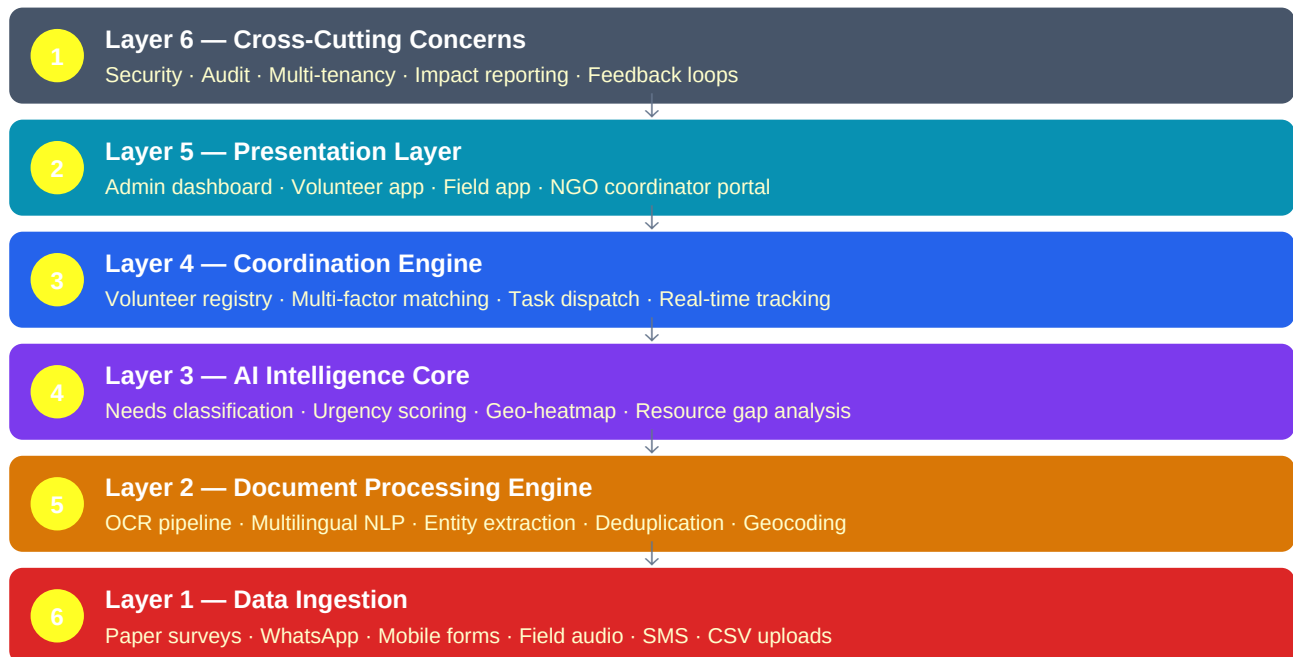
FD-6 — Longitudinal Blindness

Without a persistent household record aggregating past needs, task outcomes, and resolution history over time, it is impossible to measure whether assistance is actually breaking the cycle of crisis. A family in its seventh food crisis in six months looks identical to a family in its first — without longitudinal data.

The Compounding Effect

These failure domains interact in a closed loop of dysfunction: bad data in → bad prioritization → wrong volunteer dispatched → poor outcome → no feedback → data stays bad. The household registry (Module 01) is the closure mechanism that finally breaks this loop by creating a persistent identity layer that every other module reads from and writes to.

NEXUS is organized into six functional layers arranged as a processing pipeline with bidirectional feedback loops. Each layer is a deployable subsystem; layers communicate via Kafka event streams and synchronous REST/GraphQL APIs.



↑ Bidirectional feedback flows at every layer interface. Outcome data from Layer 4 propagates back through Layer 3 to continuously recalibrate urgency models. The Household Registry (Module 01) is a horizontal service accessed by Layers 2, 3, and 4.

Architectural Principles

Event-Driven Core	All state changes are Kafka events. No module polls another. This enables replay, audit, and independent scaling.
Human-in-the-Loop by Design	Algorithms produce ranked suggestions; coordinators make decisions. Automation expands only after measured accuracy.
Offline-First Mobile	Field and volunteer apps use WatermelonDB (SQLite) for local-first sync. Connectivity is unreliable in target areas.
Privacy by Architecture	PII lives in a dedicated encrypted vault. Analytics and matching operate on anonymized tokens only.
Multi-Tenant Isolation	Row-level security in PostgreSQL. Each NGO is a tenant. Data is never commingled without explicit consent-gated linking.
Language-First	Every user-facing surface supports Hindi, English, Marathi, Tamil, and Bengali at minimum. Not just UI translation — data entry too.

The entry boundary of NEXUS. Accepts all data formats without requiring senders to change their behavior. Every input is treated as untrusted and unstructured until proven otherwise.

■ Paper surveys	Photographed with field app camera, uploaded to S3, triggers OCR pipeline
■ WhatsApp / SMS	Twilio webhook receives messages, classifies as text/voice/image, routes accordingly
■ Mobile forms	Offline-capable structured forms built in field app, sync when connected
■ Voice notes	Google Cloud Speech-to-Text with language detection → text → NLP pipeline
■ CSV bulk upload	NGO coordinators upload existing spreadsheets, schema mapping UI handles column alignment
■ External API	Third-party NGO systems POST to the ingestion REST endpoint directly

Transforms raw, heterogeneous input into a standardized, geocoded, deduplicated NeedRecord. This is the most complex processing layer, running as a Python microservice subscribed to ingestion Kafka events.

■ OCR Engine	Google Cloud Vision API (primary, handles handwritten multilingual text) + Tesseract (fallback, self-hosted). Images preprocessed with OpenCV: deskew, contrast enhancement, noise removal.
■ Speech Transcription	Google Cloud Speech-to-Text. Language auto-detected from audio. Transcript enters NLP pipeline identically to text inputs.
■ NLP Extraction	spaCy with custom-trained NER models. Extracts: location entities (including informal names), need category, severity signals, beneficiary descriptors, household identifiers.
■ Geocoding	Google Maps Geocoding API for formal addresses. Custom landmark lookup table for informal location identifiers (area names, ward codes, local landmarks). Maintained by field coordinators.
■ Deduplication	Calls Household Registry Matching API. Returns household_id (confirmed), provisional_id (pending review), or signals new household creation.
■ Human Review Queue	Records below confidence threshold 0.75 are routed to a coordinator-facing review interface. Low-confidence NLP outputs are flagged with reasoning.

Transforms the stream of verified NeedRecords into actionable prioritized intelligence. Runs continuously as an event-driven Python service, publishing to the coordinator dashboard in near-real-time.

■ Need Classification	Fine-tuned multilingual BERT (mBERT) classifies needs into category + subcategory taxonomy. Trained on domain-specific humanitarian corpus annotated by field experts.
■ Urgency Scoring	Weighted composite model: $U = w1 \times \text{severity} + w2 \times \text{recency} + w3 \times \text{vulnerability_multiplier} + w4 \times \text{unmet_duration} + w5 \times \text{source_reliability} - w6 \times \text{resource_coverage_in_area}$. Weights configurable per-tenant.
■ Chronic Risk Flagging	Household-level crisis_frequency (needs/month, 6-month rolling) adds urgency floor for chronically at-risk households, preventing their deprioritization during low-severity reporting periods.
■ Geo-Heatmap Aggregation	Elasticsearch geo_point aggregations cluster needs by ward and category. Published to dashboard WebSocket in real time. Drives the resource gap analysis surface.
■ Priority Queue	Top-N urgent tasks ranked and held. Coordinator interface surfaces these with full context. Queue recalculates on every new need record and on time-decay schedule.

The operational heart of NEXUS. Manages the full lifecycle of matching a volunteer to a need, dispatching them, tracking the task, and capturing the outcome.

■ Volunteer Candidate Pool	Real-time query against Volunteer Registry filtered by: PostGIS radius, availability calendar, burnout risk score below threshold.
■ Multi-Factor Matching	Cosine similarity between need requirement vector and volunteer capability vector. Dimensions: skill fit, proximity, language match, cultural context, past performance, household relationship.
■ Match Suggestion	Top 3 candidates returned with scores and reasoning. Coordinator approves or overrides. Full automation is Phase 4+ only, after accuracy is validated.
■ Task Dispatch	Pushes task briefing through best available channel: push notification (app) → WhatsApp → SMS. Two-way response parsing handles accept/decline without app.
■ Real-Time Tracking	Volunteers with app send location ping every 5 min when active. Node.js WebSocket broadcasts to coordinator dashboard. Task timeline maintained.
■ Outcome Collection	Volunteer completes structured report in app: need resolution status, materials used, new needs observed, follow-up required. Feeds back to urgency engine and household history.

Four distinct user-facing surfaces, each designed for a specific role and context. All built on the same underlying API; role-based access control determines what each surface can read or write.

■ Coordinator Dashboard (Web)	Next.js + Mapbox GL JS. Live heatmap of needs by ward and category. Priority queue with one-click match suggestions. Task tracker. Cross-org view (with linking consent). Funder impact reports.
■ Field Worker App (Mobile)	React Native + WatermelonDB. Offline-capable. Survey creation, photo capture, voice note recording. Household lookup and registration. Available in 5 regional languages.
■ Volunteer App (Mobile)	React Native. Task notifications and briefings. Navigation to household. Offline outcome report form. Personal stats and impact summary.
■ NGO Coordinator Portal	Web interface for organization-level admin: volunteer onboarding, urgency weight configuration, cross-tenant linking approvals, analytics and CSV/PDF export.

System-wide services that every other layer depends on. Not a layer in the processing sense — rather a set of horizontal services implemented once and consumed by all.

■ Authentication & Authorization	JWT + RBAC. Roles: super_admin, org_admin, coordinator, field_worker, volunteer, read_only. Row-level security in PostgreSQL enforces tenant data isolation at DB level.
■ Audit Trail	Every state-changing API call is logged with: user_id, timestamp, before/after payload snapshot, IP. Immutable append-only log. Required for DPDP compliance and funder accountability.
■ Multi-Tenancy	Tenant scoping on all data queries. Configurable: visible geography, volunteer pool sharing policy, urgency weight presets, branding, cross-tenant linking opt-in.
■ Observability Stack	Prometheus metrics, Grafana dashboards, Jaeger distributed tracing, ELK for application logs. SLA alert: p99 API latency < 500ms, ingestion lag < 30s.
■ Disaster Recovery	PostgreSQL multi-AZ with automated snapshots (RTO 4h, RPO 1h). Kafka replication factor 3. S3 raw input bucket versioning enabled for immutable audit record.

The Household Registry is not a supporting module — it is the substrate on which NEXUS becomes a system that serves communities rather than merely processing events. Without it, every need is a transaction. With it, every need is a chapter in a household's story. It is architecturally the most complex module because it must resolve the identity of people who resist easy identification: households with no formal addresses, changing family compositions, and multiple NGO relationships.

Why This Module Is Fatal Without

- Deduplication is incomplete — `household_id` FK in `NeedRecord` is a placeholder; without a master registry, fuzzy matching alone cannot reliably resolve 'family near the temple' to an existing record.
- Longitudinal impact is invisible — no way to know if a household's situation improves over time without aggregated history of all needs, tasks, and outcomes per household.
- Cross-NGO duplication persists — two NGOs serving the same household each maintain separate records with no linkage, continuing duplicate resource allocation.
- DPDP consent cannot be enforced — consent must be tied to a beneficiary entity, not just a need record. Opt-outs and deletion requests require a household-level consent store.
- Vulnerability flags are ephemeral — flags attached to `NeedRecord` are re-extracted on every report. A household with a disabled elderly member should carry that flag persistently.
- Chronic risk is invisible — `crisis_frequency` (needs per month) can only be computed if needs are aggregated against a persistent household entity over time.

Internal Service Architecture

The Household Registry is deployed as a dedicated microservice with its own PostgreSQL + PostGIS database, Redis cache layer, and PII vault (separate encrypted store). It publishes to and subscribes from a dedicated set of Kafka topics.

Service	Implementation	Responsibilities
Registry API Gateway	REST + GraphQL	Auth enforcement, tenant scoping, rate limiting, field-level access control based on consent scope
Identity Resolution Engine	Python service	4-stage probabilistic matching cascade. Handles exact match, geospatial cluster, fuzzy attribute match, composite scoring
Profile Manager	CRUD service	Household and member CRUD with full version history. Every field change creates an immutable audit entry
Consent Engine	DPDP-compliant service	Consent collection, scope enforcement, TTL management, opt-out cascade, right-to-erasure workflow
Cross-Tenant Linker	Consent-gated bridge	Three-party consent model (household + org A + org B). Field-level sharing via virtual views, never data copies
Merge/Demerge Service	Transactional workflow	Coordinator-driven merge of duplicate records. History ledger preserved. All FK references rewritten atomically
History Ledger	Append-only PostgreSQL	Immutable event log of all need reports, task outcomes, member changes, consent events per household

Table: households — Canonical Master Record

Field Name	Type / Enum	Notes / Constraints
household_id	UUID PK	Global, immutable, system-generated on first record creation
global_household_id	UUID NULLABLE	Cross-tenant shared identity token. Null until cross-org linking approved
tenant_id	UUID FK → tenants	Owning organization
ward_id	VARCHAR FK	Administrative unit. Used for area-level aggregation and analytics
location	GEOGRAPHY(POINT,4326)	PostGIS point. Centroid of dwelling. Required for geospatial queries
location_confidence	FLOAT(0–1)	GPS fix=1.0 · geocoded address=0.7 · landmark estimate=0.4
location_description	TEXT	Informal description preserved verbatim. Never overwritten by geocoder
landmark_tags	TEXT[]	['near dargah', 'red door', 'corner of main road'] — searchable array
dwelling_type	ENUM	permanent / semi-permanent / temporary / open-space
total_members	INT	Denormalized count. Updated by trigger on household_members changes
vulnerability_score	FLOAT(0–1)	Computed from member profiles. Recomputed on any member change
vulnerability_flags	JSONB	Persistent: {has_child, has_elderly, has_disabled, has_pregnant, chronic_illness, single_parent}
economic_tier	ENUM	below_poverty / marginal / low / medium — set by field worker assessment
total_needs_reported	INT	Denormalized counter. Incremented on each NeedRecord.household_id bind
total_tasks_completed	INT	Denormalized counter. Incremented on Task outcome = completed
last_need_reported_at	TIMESTAMPTZ	
last_assistance_at	TIMESTAMPTZ	
total_assistance_value	DECIMAL	Estimated value of all assistance received. Used in funder reports
assistance_categories	TEXT[]	Running list of categories served. Enables gap analysis
crisis_frequency	FLOAT	Needs per month, rolling 6-month average. Key chronic-risk signal
status	ENUM	active / relocated / dissolved / merged_away / opted_out
merged_into	UUID FK NULLABLE	Points to surviving record when this record was merged away
data_quality_score	FLOAT(0–1)	Completeness × confidence. Used to prioritize review queue
created_at / updated_at	TIMESTAMPTZ	
created_by	UUID FK → users	Field worker who created the record

Table: household_members — Individual Demographics

Field Name	Type / Enum	Notes / Constraints
member_id	UUID PK	

Field Name	Type / Enum	Notes / Constraints
household_id	UUID FK → households	
role_in_household	ENUM	head / spouse / child / parent / dependent / other
age_bracket	ENUM	infant / child / youth / adult / elderly — not exact age (privacy)
gender	ENUM NULLABLE	Recorded only with explicit consent
is_primary_contact	BOOL	Only one member per household can be primary contact
vulnerability_flags	JSONB	Individual: {disabled, chronic_illness, pregnant, malnourished, mental_health}
pii_ref	UUID → PII vault	Encrypted pointer to name, phone, photo. Never stored inline
is_present	BOOL	Currently resident. Allows seasonal worker tracking
added_at	TIMESTAMPTZ	
removed_at	TIMESTAMPTZ NULLABLE	Soft delete. Preserves historical composition

Table: household_consent — Legal & Ethical Backbone

Field Name	Type / Enum	Notes / Constraints
consent_id	UUID PK	
household_id	UUID FK → households	
primary_contact_ref	UUID FK → household_members	Which member granted consent
consent_type	ENUM	data_collection / location_sharing / cross_org_linking / analytics / photo
scope	JSONB	{tenants_allowed: [], purposes: [], data_fields_allowed: []}
granted_at	TIMESTAMPTZ	
expires_at	TIMESTAMPTZ	Consent has TTL. Requires renewal. Configurable per consent_type
revoked_at	TIMESTAMPTZ NULLABLE	Null if still active
revocation_reason	TEXT	
collected_by	UUID FK → users	Field worker responsible. Required for accountability
collection_method	ENUM	verbal_witnessed / signed_form / digital_app
language_used	VARCHAR	Language in which consent was explained to household
opt_out	BOOL	System-wide opt-out flag. Triggers PII deletion cascade
opt_out_scope	TEXT[]	Specific purposes opted out of — allows partial consent retention

Table: household_cross_tenant_links — Cross-Organization Bridge

Field Name	Type / Enum	Notes / Constraints
link_id	UUID PK	
household_id_tenant_a	UUID FK → households	
household_id_tenant_b	UUID FK → households	
global_household_id	UUID	Shared identity token assigned at link creation
link_confidence	FLOAT	Algorithm confidence this is the same physical household
link_method	ENUM	auto_matched / coordinator_confirmed / household_confirmed
link_status	ENUM	proposed / active / rejected / revoked
consent_id_a / consent_id_b	UUID FK → consent	Both organizations must have valid cross_org_linking consent
fields_shared	TEXT[]	Which fields each org can see. Virtual view — never a data copy
approved_at	TIMESTAMPTZ	
approved_by	UUID FK → users	Super-admin or both org coordinators depending on config

Table: household_history — Immutable Longitudinal Ledger

Field Name	Type / Enum	Notes / Constraints
event_id	UUID PK	
household_id	UUID FK → households	

Field Name	Type / Enum	Notes / Constraints
event_type	ENUM	need_reported / need_resolved / task_dispatched / task_completed / member_added / member_removed / vulnerability_updated / consent_changed / assistance_received / location_updated / merged / linked
event_timestamp	TIMESTAMPTZ	
event_payload	JSONB	Full snapshot of relevant state at time of event
related_need_id	UUID FK NULLABLE	
related_task_id	UUID FK NULLABLE	
triggered_by	UUID FK → users	

This table is append-only. No UPDATE or DELETE operations are permitted. Enforced at database level via trigger + row-level security policy. The PII erasure right-to-erasure workflow anonymizes event_payload fields without deleting the row, preserving the audit trail while removing personal data.

Identity Resolution Engine — 4-Stage Cascade

The identity resolution engine answers: 'Is the household described in this new need report the same as an existing household in the registry?' It runs four escalating stages, stopping at the first confident match.

	Exact Signal Match	Confidence: 0.95+	→ AUTO LINK
Phone number match · unique ID present · household_id already known from prior session. If any exact signal matches, confidence ≥ 0.95, auto-link immediately without further stages.			
	Geospatial Cluster	Confidence: varies	→ FILTER CANDIDATES
PostGIS query: find all households within 50m of the reported location. Filter by family size (± 2 members) and dwelling type. Returns a scored candidate list for Stage 3.			
	Fuzzy Attribute Match	Confidence: 0.55–0.94	→ SCORE + ROUTE
Jaccard similarity on landmark_tags · vulnerability profile cosine similarity · description embedding cosine score (using sentence-transformers). Score 0.80–0.94 → coordinator review. Score < 0.55 → new household.			
		Confidence: final	→ DISPATCH DECISION
Weighted ensemble of all stage signals. Final confidence score routes to: ≥ 0.80 auto-link with flag · 0.55–0.79 human review queue · < 0.55 create new household. Coordinator decisions feed back into model training.			

Stateless, horizontally scalable entry point. Accepts all data formats, validates schema, routes to appropriate processor, stores raw input to S3 immutably, emits Kafka event.

Input channels	HTTP REST (mobile app, CSV upload, external API) · Twilio webhook (SMS, WhatsApp) · internal triggers from document scanner
Format detection	MIME type + content heuristic. Routes: image→OCR queue · audio→STT queue · structured JSON→bypass NLP · CSV→schema mapper
Idempotency	Each raw input gets a content hash. Duplicate submissions (re-uploads, network retries) are detected and rejected before processing.
Failure handling	Dead letter queue for failed routing. S3 stores every raw input regardless of downstream failure. Nothing is lost at the gateway.
Rate limiting	Per-tenant and per-source-channel rate limits. Prevents rogue CSV uploads from saturating the NLP pipeline.

Table: raw_inputs

Field Name	Type / Enum	Notes / Constraints
raw_input_id	UUID PK	
tenant_id	UUID FK	
source_type	ENUM	paper_photo / whatsapp_text / whatsapp_voice / whatsapp_image / mobile_form / sms / csv / api
source_identifier	VARCHAR	Phone number, API key ID, or app session ID
content_hash	VARCHAR	SHA-256 of raw content. Deduplication key
s3_key	VARCHAR	Immutable storage reference
processing_status	ENUM	queued / processing / completed / failed / duplicate
received_at	TIMESTAMPTZ	
processed_at	TIMESTAMPTZ NULLABLE	

Python microservice. Subscribes to Kafka ingestion events. Orchestrates the full OCR → transcription → NLP → geocoding pipeline. Produces candidate NeedRecords.

OCR preprocessing	OpenCV pipeline: deskew (Hough transform) → denoise (fastNIMeans) → contrast CLAHE → binarize (Otsu). Applied before sending to Vision API.
Confidence scoring	Each field extracted from OCR/NLP gets an individual confidence score. Record-level confidence = minimum field confidence. Threshold 0.75 for auto-accept.
Language detection	langdetect library on extracted text. Routes to language-specific NLP model. Supports: en, hi, mr, ta, bn, te, gu.
Fallback chain	Google Vision fails → Tesseract local. Tesseract confidence < 0.5 → route to human review. Voice transcription fails → route to manual entry.
Scaling	Kubernetes HPA on Kafka consumer lag metric. Scales out processing pods when ingestion queue depth exceeds threshold.

Table: processing_jobs

Field Name	Type / Enum	Notes / Constraints
job_id	UUID PK	
raw_input_id	UUID FK → raw_inputs	
stage	ENUM	ocr / transcription / nlp / geocoding / household_match / completed
ocr_text	TEXT	Raw OCR output before NLP
ocr_confidence	FLOAT	
detected_language	VARCHAR	
nlp_entities	JSONB	Extracted entities with individual confidence scores
geocode_result	JSONB	lat, lng, confidence, method used
candidate_need_id	UUID FK NULLABLE	Points to NeedRecord once created
review_reason	TEXT	Populated when routed to human review
started_at / completed_at	TIMESTAMPTZ	

Extracts structured meaning from normalized text. The domain-specific NER and classification models are the core IP of this service. Requires expert annotation investment upfront.

Need classification	Fine-tuned mBERT. 12 top-level categories: food, health, shelter, water, education, livelihood, sanitation, mental_health, legal, documentation, mobility, other. Each has subcategory taxonomy.
NER — location	Custom NER model trained on informal Indian location descriptions. Recognizes: street landmarks, area names, ward references, directional descriptions ('third lane after the pump').
NER — severity	Severity signals extracted from language patterns. 'Hasn't eaten in three days' → severity 0.95. 'Running low on supplies' → 0.45. Language-specific pattern library per supported language.
NER — beneficiary	Extracts count and descriptor: '3 children', 'elderly woman', 'pregnant'. Maps to household_members vulnerability flags for persistent storage.
Active learning	Low-confidence classification decisions are surfaced to domain expert annotators. Corrections are collected and used in quarterly model retraining cycles.
Model versioning	MLflow tracks all model versions. Champion/challenger deployment via Kubernetes canary. Rollback in < 5 minutes if accuracy drops.

Computes and continuously updates the urgency score U for every active NeedRecord. This score is the primary ordering signal for the coordinator's priority queue.

Formula	$U = w1 \times \text{severity_score} + w2 \times \text{recency_factor} + w3 \times \text{vulnerability_multiplier} + w4 \times \text{unmet_duration} + w5 \times \text{source_reliability} - w6 \times \text{resource_coverage_in_area}$
Chronic risk floor	Households with <code>crisis_frequency > 2.0</code> (more than 2 needs/month average) receive a minimum urgency floor of 0.60 regardless of reported severity. Prevents chronic households from being perpetually deprioritized.
Time decay	Needs that have not been assigned within 24h have their urgency score escalated by +0.05/hour until assigned or closed. Ensures nothing stays unaddressed indefinitely.
Resource coverage penalty	The $w6$ term subtracts urgency in areas already well-resourced. Prevents pile-on in easily accessible communities while harder-to-reach areas remain underserved.
Tenant customization	Weights $w1$ – $w6$ are configurable per tenant. A child nutrition NGO sets $w3$ (vulnerability) highest. A general relief org uses balanced defaults. Super-admin enforces valid weight ranges.
Score persistence	Urgency scores stored in Redis for millisecond access. Persisted to PostgreSQL every 5 minutes for analytics. Recomputed on every relevant Kafka event.

Table: `urgency_scores`

Field Name	Type / Enum	Notes / Constraints
<code>score_id</code>	UUID PK	
<code>need_id</code>	UUID FK → <code>need_records</code>	
<code>household_id</code>	UUID FK → <code>households</code>	
<code>score</code>	FLOAT(0–1)	Current urgency score
<code>component_scores</code>	JSONB	{severity, recency, vulnerability, duration, reliability, coverage}
<code>weight_preset_id</code>	UUID FK	Which tenant weight configuration was used
<code>chronic_risk_floor_applied</code>	BOOL	
<code>time_decay_escalations</code>	INT	Number of 24h escalation cycles applied
<code>computed_at</code>	TIMESTAMPTZ	
<code>expires_at</code>	TIMESTAMPTZ	When score must be recomputed (max 1h)

Manages volunteer identity, capabilities, availability, location, and wellbeing. The data this module maintains is the input to the Matching Engine.

Profile management	Full volunteer lifecycle: registration, verification (org coordinator approves), onboarding, deactivation. Skills taxonomy maintained as controlled vocabulary, not free text.
Availability calendar	Volunteers set recurring availability and one-time overrides. Calendar stored as JSONB rules + exception dates. API returns 'is_available_at(volunteer_id, datetime)' boolean.
Real-time location	App sends location ping every 5 minutes when volunteer is active. Stored in Redis (live) and PostgreSQL (history). Used for proximity matching and arrival tracking.
Burnout risk scoring	$B = \text{deployments_last_7d} / \text{max_weekly_capacity} \times 0.4 + (1 - \text{avg_outcome_rating}) \times 0.3 + \text{consecutive_days_active} / 7 \times 0.3$. $B > 0.8 \rightarrow$ excluded from auto-matching, coordinator alerted.
Skills verification	Skill claims require evidence: medical skills need license number, legal skills need bar registration, first aid needs cert upload. Unverified skills are flagged in match scoring.

Table: volunteers

Field Name	Type / Enum	Notes / Constraints
volunteer_id	UUID PK	
tenant_id	UUID FK	Owning organization
pii_ref	UUID $\rightarrow$ PII vault	Name, phone, email, photo
location_home	GEOGRAPHY(POINT,4326)	Registered home location
location_current	GEOGRAPHY(POINT,4326)	Last known live location from app ping
location_updated_at	TIMESTAMPTZ	
skills	JSONB	[{skill: 'medical', verified: true, credential_ref: '...'}, ...]
languages	TEXT[]	Languages spoken. ISO 639-1 codes
availability_schedule	JSONB	Recurring rules + exception dates
max_distance_km	INT	Maximum willingness to travel per task
total_deployments	INT	
outcome_rating_avg	FLOAT	Rolling average of task outcome ratings
burnout_risk_score	FLOAT	Recomputed daily and on each task completion
consecutive_active_days	INT	
active	BOOL	Currently accepting task assignments
verified	BOOL	Approved by org coordinator
ngo_affiliation	UUID FK NULLABLE	

Produces ranked volunteer candidate lists for coordinator review. Matching is multi-dimensional, combining geospatial filtering with capability vector similarity and wellbeing constraints.

Input processing	Receives task requirement spec: need category, required skills, location, language preference, urgency level, household context (from registry).
Geospatial pre-filter	PostGIS query: volunteers within max_distance_km of task location AND currently active AND burnout_risk < 0.8. Produces candidate pool (typically 5–20 volunteers).
Capability vector	Each volunteer has a capability vector: skill dimensions (0/0.5/1 for absent/unverified/verified) + language dimensions + proximity score + performance rating. Cosine similarity with need requirement vector.
Household context boost	If the household has a prior successful task with a specific volunteer, that volunteer receives a +0.15 similarity boost. Language match is hard required if household has a language flag.
Candidate ranking	Top 3 candidates returned with full score breakdown: base similarity, household boost, proximity score, availability confidence, burnout risk level. Coordinator sees reasoning.
Override learning	When coordinator overrides a suggestion, the override reason is captured. Patterns in overrides feed quarterly model recalibration.

Table: match_requests

Field Name	Type / Enum	Notes / Constraints
match_id	UUID PK	
task_id	UUID FK → tasks	
need_id	UUID FK → need_records	
household_id	UUID FK → households	
requirement_vector	JSONB	Normalized capability requirements
candidate_pool_size	INT	How many volunteers passed the geospatial pre-filter
top_matches	JSONB	[{volunteer_id, similarity_score, score_breakdown, rank}, ...]
selected_volunteer_id	UUID FK NULLABLE	Who was actually assigned
selection_method	ENUM	algorithm_top / coordinator_override / manual
override_reason	TEXT NULLABLE	
computed_at	TIMESTAMPTZ	

Manages the full task lifecycle state machine. Central coordinator between the Matching Engine, Notification Service, and Volunteer App.

State machine	unassigned → dispatched → accepted → in_progress → [completed cancelled] → [needs_reassignment]. Every transition emits a Kafka event and a notification.
Acceptance timeout	Volunteer has configurable window (default 15 min) to accept. On timeout, automatically dispatches to next match candidate. Three rejections → escalate to coordinator.
Task briefing	Structured JSON briefing: household location (with navigation link), need description, materials to bring, special instructions from coordinator, household vulnerability flags (shared only if consent permits).
Reassignment	Cancelled tasks re-enter matching engine with priority boost. If no volunteer available within 1h, coordinator is alerted for manual escalation.
Multi-task coordination	System tracks volunteer's active task count. Same volunteer not assigned second task until current task reaches in_progress status, preventing briefing overload.

Table: tasks

Field Name	Type / Enum	Notes / Constraints
task_id	UUID PK	
need_id	UUID FK → need_records	
household_id	UUID FK → households	
match_id	UUID FK → match_requests	
assigned_volunteer_id	UUID FK NULLABLE	
status	ENUM	unassigned / dispatched / accepted / in_progress / completed / cancelled / needs_reassignment
dispatch_attempts	INT	Number of volunteers notified before acceptance
accepted_at / started_at / completed_at	TIMESTAMPZ	
outcome_status	ENUM	need_fully_resolved / need_partially_met / unresolved / escalated
outcome_notes	TEXT	Volunteer's free-text report
materials_provided	JSONB	{food_kg: 5, medicines: ['...'], ...}
new_needs_observed	JSONB	Additional needs surfaced during visit → creates new NeedRecords
follow_up_required	BOOL	
follow_up_date	DATE NULLABLE	
volunteer_rating	FLOAT NULLABLE	Coordinator rating of volunteer performance on this task
household_rating	FLOAT NULLABLE	Household satisfaction (if consent permits collection)

Routes outbound communications through the appropriate channel for each recipient. Handles template management, two-way response parsing, and delivery confirmation.

Channel priority	Push notification (app installed, active) → WhatsApp (registered number) → SMS (universal fallback). System tracks which channel last succeeded per recipient.
Template engine	Handlebars templates per notification type × per language. Variables injected at send time. Templates reviewed by domain experts for clarity and cultural appropriateness.
Two-way SMS	Volunteers without the app respond via SMS keywords: '1' = ACCEPT, '2' = DECLINE, '3' = CALL ME. Parser converts to task state events. Ambiguous responses trigger callback.
WhatsApp bot	Meta Business API. Structured message with quick reply buttons for accept/decline. Richer briefing cards with embedded map link.
Delivery tracking	Each message gets a delivery_id. Webhooks from Twilio/Meta update delivery status. Undelivered messages trigger channel failover after 2-minute timeout.
Grievance channel	Special inbound keyword 'COMPLAINT' creates a grievance record even for opted-out households. Maintained as a permanent accessible channel for data rights.

Aggregates operational data into impact intelligence. Reads from PostgreSQL read replica and Elasticsearch analytics index. Never touches the operational primary database.

Impact metrics	Primary KPIs: needs volume by category/ward/time · resolution rate by category · avg time-to-assignment · avg time-to-resolution · household improvement index · volunteer utilization rate · burnout incidents.
Household improvement index	Per household: ratio of resolved-to-unresolved needs over rolling 90 days. Aggregated at ward level. Primary funder-facing metric. Shows whether assistance is actually breaking the crisis cycle.
Cross-org deduplication report	Uses global_household_id to count households served by multiple NGOs in same period, by ward. Actionable signal for coordination conversations between organizations.
Funder reports	Configurable report templates. Export as PDF or CSV. Includes: geography served, beneficiary count, need categories addressed, outcome rates, volunteer hours, estimated assistance value.
Predictive signals	Phase 5 feature: seasonal pattern detection (certain need categories spike predictably), event-triggered alerts (festival season → food demand increase). Surfaces anticipated needs before they are reported.

Manages the organizational layer of NEXUS. Each NGO or social group is a tenant. This service controls what each tenant can see, do, and share.

Tenant configuration	Visible geography (ward/district boundary), volunteer pool policy (private or shared), urgency weight presets, branding colors/logo, language default, cross-tenant linking policy.
Super-admin functions	Approve new tenant onboarding, review cross-tenant link proposals, configure system-wide urgency weight bounds, manage domain vocabulary (need categories, skill taxonomy), monitor system health.
Cross-tenant linking policy	Per-tenant-pair configuration: auto-propose (system suggests, coordinator approves) · coordinator-only (no auto-proposals) · disabled. Stored in tenant_pair_policies table.
Volunteer pool sharing	Two orgs can agree to share volunteer pools: volunteers of org A are visible to org B for task assignment (with volunteer's explicit consent). Useful for coalition responses.
Data retention policies	Configurable per tenant: active need retention, resolved need archival schedule, anonymization timeline. Must comply with DPDP minimum standards but can be stricter.

Table: tenants

Field Name	Type / Enum	Notes / Constraints
tenant_id	UUID PK	
org_name	VARCHAR	
org_type	ENUM	ngo / community_group / government / coalition
geography_boundary	GEOGRAPHY(POLYGON, 4326)	Visible area for this tenant
urgency_weights	JSONB	w1–w6 weight configuration
linking_policy	ENUM	auto_propose / coordinator_only / disabled
volunteer_sharing_opt_in	BOOL	
default_language	VARCHAR	ISO 639-1
active	BOOL	
onboarded_at	TIMESTAMPTZ	
data_protection_officer_ref	UUID → PII vault	DPO contact for DPDP grievance routing

need_records — Central Event Entity

Field Name	Type / Enum	Notes / Constraints
need_id	UUID PK	
source_type	ENUM	paper / whatsapp / mobile / csv / api / field_observation
raw_input_ref	UUID FK → raw_inputs	Immutable link to original unprocessed input
processing_job_id	UUID FK → processing_jobs	
reported_by	UUID FK → users	Field worker who submitted
reported_at	TIMESTAMPTZ	When the need was originally observed (not upload time)
household_id	UUID FK → households	Resolved by Household Matching API
household_resolution_confidence	FLOAT	Matching engine confidence at time of resolution
location	GEOGRAPHY(POINT,4326)	Resolved from OCR/NLP, may differ from household centroid
ward_id	VARCHAR FK	
category	ENUM	food / health / shelter / education / livelihood / water / sanitation / mental_health / legal / documentation / mobility / other
subcategory	VARCHAR	Domain-specific subcategory from taxonomy
description	TEXT	Normalized description, language-standardized
original_description	TEXT	Preserved verbatim from source
detected_language	VARCHAR	ISO 639-1
severity_score	FLOAT(0–1)	NLP-extracted severity from language signals
urgency_score	FLOAT(0–1)	Computed by Urgency Scoring Engine. Refreshed continuously
beneficiary_count	INT	
vulnerability_flags	JSONB	Extracted at report time. Persisted to household on verified=true
status	ENUM	unverified / verified / assigned / in_progress / resolved / closed / duplicate
verified_by	UUID FK → users NULLABLE	
verified_at	TIMESTAMPTZ NULLABLE	
assigned_task_id	UUID FK → tasks NULLABLE	
resolution_notes	TEXT NULLABLE	
is_chronic	BOOL	Set to true if household crisis_frequency > 2.0
created_at / updated_at	TIMESTAMPTZ	

Entity Relationship Overview

The following table summarizes all primary entities, their primary keys, and their key foreign relationships. PostGIS spatial indexes exist on all GEOGRAPHY columns. Composite indexes exist on (tenant_id, status), (ward_id, category, urgency_score DESC), and (household_id, created_at DESC) for the primary query patterns.

Entity	Primary Key	Key FKs	Purpose
tenants	tenant_id	—	Root entity. All others scope to tenant_id
households	household_id	tenant_id, ward_id	Master registry. Linked by global_household_id cross-tenant
household_members	member_id	household_id	Individual demographics. Soft-deleted on departure
household_consent	consent_id	household_id, member_id	Purpose-scoped, TTL-bearing consent records
household_cross_tenant_links	link_id	household_id×2, consent_id×2	Three-party consent. Virtual view sharing
household_history	event_id	household_id, need_id, task_id	Append-only immutable ledger
raw_inputs	raw_input_id	tenant_id	Immutable S3-backed original inputs
processing_jobs	job_id	raw_input_id, need_id	Pipeline stage tracking
need_records	need_id	household_id, ward_id, task_id	Central event entity
urgency_scores	score_id	need_id, household_id	Redis-cached, Postgres-persisted
volunteers	volunteer_id	tenant_id	Capability + availability + wellbeing
match_requests	match_id	task_id, need_id, household_id	Matching engine input/output log
tasks	task_id	need_id, household_id, volunteer_id, match_id	Operational coordination unit
tenants_pair_policies	policy_id	tenant_id×2	Cross-org linking and sharing configuration

Component	Technology	Justification
Backend API	Python + FastAPI	Async-native, Pydantic schema validation, excellent ML ecosystem, high developer availability in India, auto-generated OpenAPI docs
Primary Database	PostgreSQL 16 + PostGIS	ACID transactions prevent double-booking. PostGIS handles all geospatial queries. Row-level security enforces multi-tenancy. Proven at NGO scale.
Document Store	MongoDB Atlas	Schema-free. Handles wild variability of raw ingestion records before normalization. TTL indexes auto-expire old processing jobs.
Search & Analytics	Elasticsearch 8	geo_point aggregations for heatmap. Multilingual full-text search on need descriptions. Near-real-time analytics index refresh.
Message Queue	Apache Kafka (AWS MSK)	Durable event streaming. Decouples all services. Consumer group isolation. Replay capability for audit. Scales to 100k events/day easily.
Cache Layer	Redis 7	Urgency score hot cache (< 1ms reads). Volunteer live location. Rate limit counters. Distributed locks for task assignment (prevent double-dispatch).
OCR Primary	Google Cloud Vision API	Best-in-class for handwritten multilingual text including Devanagari, Tamil scripts. Handles field photos with variable lighting.
OCR Fallback	Tesseract 5 (self-hosted)	Zero marginal cost. Handles printed text reliably. Kubernetes sidecar. Used when Vision API is unavailable or budget threshold hit.
Image Processing	OpenCV 4	Deskew, denoise, contrast enhancement before OCR. Runs as preprocessing step in document processing service pods.
NLP Framework	spaCy + mBERT (HuggingFace)	spaCy for NER pipeline. Fine-tuned mBERT for need classification and severity scoring. MLflow for model versioning and experiment tracking.
STT (Voice)	Google Cloud Speech-to-Text	Language auto-detection. Handles 7 Indian languages. Speaker diarization not required (single speaker field notes).
Mobile Apps	React Native + WatermelonDB	Single codebase iOS + Android. WatermelonDB provides SQLite-backed offline-first sync. Critical for rural connectivity gaps.
Coordinator Dashboard	Next.js 14 + Mapbox GL JS	SSR for fast initial load. Mapbox handles vector tile rendering for ward-level heatmaps. WebSocket via Socket.io for live updates.
Real-Time Layer	Node.js + Socket.io	Event loop optimized for high-concurrency WebSocket connections. Decoupled from FastAPI business logic layer.
SMS Channel	Twilio Programmable SMS	Universal fallback channel. Two-way keyword parsing. Delivery webhooks. India SMS regulatory compliance built in.
WhatsApp Channel	Meta Business API	Structured message templates with quick reply buttons. Webhook-driven. Requires Meta BSP registration (allow 4–6 weeks).
Infrastructure	AWS (primary)	RDS for PostgreSQL, MSK for Kafka, S3 for raw inputs, EKS for Kubernetes, KMS for PII vault encryption, CloudFront for static assets.
Container Orchestration	Kubernetes (EKS)	HPA on Kafka consumer lag (processing pods) and RPS (API pods). Resource quotas per namespace. Rolling deployments.
CI/CD	GitHub Actions + ArgoCD	GitOps deployment. ArgoCD syncs Kubernetes state from Git. Canary deployments for NLP model updates via Argo Rollouts.
Monitoring	Prometheus + Grafana + Jaeger	Service metrics, dashboards, distributed tracing. SLA alert: p99 API latency < 500ms, ingestion lag < 30s, matching time < 2s.

Component	Technology	Justification
Geocoding	Google Maps Geocoding API	Formal address resolution. Augmented by custom landmark lookup table (maintained by field coordinators in admin portal).
Embedding Model	sentence-transformers	all-MiniLM-L6-v2 for household description embeddings used in Stage 3 identity resolution fuzzy matching.

Internal REST API (FastAPI)

Between microservices. OpenAPI 3.1 spec auto-generated. mTLS for service-to-service authentication within the cluster. All endpoints versioned under /api/v1/.

Endpoint	Purpose	Primary Consumer
POST /api/v1/needs	Create NeedRecord from processed data	Document Processing Service
GET /api/v1/needs/priority-queue	Top-N needs by urgency score for tenant	Coordinator Dashboard
PATCH /api/v1/needs/{id}/status	Update need status (verify, close, reopen)	Coordinator, System
POST /api/v1/households/match	Run identity resolution for incoming need	Document Processing Service
GET /api/v1/households/{id}/history	Full longitudinal event history for household	Analytics, Coordinator
POST /api/v1/households/{id}/consent	Record new consent or update scope	Field App
DELETE /api/v1/households/{id}/pii	Trigger right-to-erasure workflow	Admin Service
POST /api/v1/tasks/match	Request volunteer match candidates for task	Coordination Engine
POST /api/v1/tasks/{id}/dispatch	Dispatch task to selected volunteer	Coordination Engine
PATCH /api/v1/tasks/{id}/outcome	Submit task completion report	Volunteer App
GET /api/v1/analytics/heatmap	Geo-aggregated needs by ward and category	Dashboard
GET /api/v1/volunteers/{id}/availability	Check volunteer availability at datetime	Matching Engine

External REST API — Third-Party NGO Integration

Rate-limited, API-key authenticated. Enables external NGO systems to push needs data or pull analytics without building to the internal service mesh. Webhooks available for event-driven integration.

Endpoint	Purpose	Auth
POST /ext/v1/needs/bulk	Bulk ingest needs from external system (CSV/JSON)	API key, per-tenant
GET /ext/v1/analytics/summary	Pull aggregated impact metrics for tenant	API key, read-only scope
POST /ext/v1/webhooks/register	Register a webhook URL for event callbacks	API key
GET /ext/v1/households/lookup	Check if a household exists (anonymized result only)	API key, consent-gated

Real-Time WebSocket API (Socket.io)

Powers the live coordinator dashboard. Rooms are scoped to tenant_id. Authentication via JWT upgrade on handshake. Coordinator subscribes to ward-level rooms for geofenced updates.

Event	Direction	Payload / Purpose
need.created	Server → Client	New NeedRecord created and verified. Payload: need summary + urgency score + ward

Event	Direction	Payload / Purpose
need.urgency_updated	Server → Client	Urgency score recalculated. Triggers priority queue reorder in dashboard
task.assigned	Server → Client	Task assigned to volunteer. Dashboard updates task tracker
task.status_changed	Server → Client	Any task lifecycle transition. Includes accepted, in_progress, completed
volunteer.location_updated	Server → Client	Live volunteer location ping. Used for field tracking overlay on map
household.linked	Server → Client	Cross-tenant household link approved. Dashboard updates dedup metrics
alert.escalation	Server → Client	Need unassigned > 24h, or burnout threshold exceeded
coordinator.override	Client → Server	Coordinator manually assigns a volunteer outside algorithm suggestion

Phased Implementation Roadmap

44-week delivery plan with clear module gates



44-week total timeline. Each phase gate requires UAT sign-off from NGO partner coordinators before next phase begins.

Phase 0 — Foundation · Weeks 1–6

Database schema design and migration tooling (Alembic). JWT + RBAC authentication service. Multi-tenancy framework with row-level security. CI/CD pipeline (GitHub Actions + ArgoCD). AWS environment setup (staging + production). Seed the system with anonymized test data representing realistic NGO scenarios across 5 ward profiles. Kafka cluster provisioned and topic schema defined.

Gate Milestones:

- PostgreSQL + PostGIS provisioned
- Authentication service live
- Multi-tenant framework tested
- CI/CD pipeline operational
- Monitoring stack baseline established

Phase 0.5 — Household Registry Foundation · Weeks 5–12 (overlapping)

The Household Registry is built as a prerequisite for Phase 1. Without household_id resolution, NeedRecords cannot be created in their final form. Five-table schema deployed. PII vault with AWS KMS field-level encryption. Consent engine with full DPDP audit log. Basic CRUD API with tenant scoping. Human review queue interface for coordinators to manage unresolved matches. Landmark lookup table seeded from field partner data.

Gate Milestones:

- All 5 household tables migrated
- PII vault encryption verified
- Consent API tested with DPDP checklist
- Identity resolution Stages 1–2 operational
- Human review queue interface live

Phase 1 — Core Ingestion · Weeks 7–14

Mobile form builder for field workers (no-code survey creation). CSV bulk upload with schema mapping UI. WhatsApp bot integration via Twilio. OCR pipeline for paper photos (OpenCV preprocessing + Google Vision + Tesseract fallback). Manual data review interface. Ingestion Gateway with S3 buffering and Kafka publishing. Identity resolution Stages 1–2 wired to ingestion.

Gate Milestones:

- Mobile form app functional (Android priority)

- CSV upload with column mapping
- WhatsApp bot live for 1 pilot organization
- OCR pipeline processing paper surveys
- All inputs producing NeedRecord candidates
- Household matching Stages 1–2 in production

Phase 2 — Intelligence Layer · Weeks 15–22

Domain-specific NLP model training (6 weeks of annotated corpus collection with field expert involvement required before training). mBERT fine-tuning for need classification and severity scoring. Urgency scoring engine with configurable weights. Geo-heatmap via Elasticsearch aggregations. Read-only analytics dashboard v1. Deduplication engine Stages 3–4 complete. Google Cloud Speech-to-Text for voice note ingestion.

Gate Milestones:

- NLP models trained and validated ($F1 > 0.82$)
- Urgency scoring live with 3 tenant presets
- Heatmap dashboard functional
- Identity resolution 4-stage cascade complete
- Voice note ingestion operational
- Analytics v1 with ward-level need breakdown

Phase 3 — Volunteer Coordination · Weeks 23–30

Volunteer registration and profile system. React Native volunteer app (Android priority). Basic manual matching with coordinator interface showing top-3 candidates. Notification service (push + WhatsApp + SMS). Task lifecycle state machine. Outcome collection from volunteer mobile app. Burnout risk scoring. Task reassignment workflow on volunteer decline.

Gate Milestones:

- Volunteer app live (Android)
- 300+ volunteers onboarded across 3 NGOs
- Manual matching with algorithm suggestions
- Full task lifecycle managed in system
- Outcome reports feeding household history
- Burnout alerts functioning

Phase 4 — Smart Matching · Weeks 31–38

Multi-factor capability vector matching. Real-time volunteer location integration. Household context signals in match scoring. Automatic match suggestions with coordinator approval gate. Cross-organization coordination features (volunteer pool sharing, cross-tenant need visibility with consent). Live coordinator dashboard v2 with WebSocket updates and full map overlay.

Gate Milestones:

- Cosine similarity matching replacing rule-based
- Live location tracking on coordinator map

- Household context boosting active
- Cross-org volunteer sharing with 2 partner NGOs
- Dashboard real-time updates < 2s latency
- Override learning pipeline collecting data

Phase 5 — Intelligence Feedback Loop · Weeks 39–46

Outcome data feeding back into urgency scoring calibration. Volunteer performance scoring from outcomes. Chronic risk floor based on household crisis_frequency. Predictive need detection (seasonal patterns). Full funder impact reporting with PDF export. External REST API for third-party NGO integration. Full DPDP compliance audit. Right-to-erasure workflow tested end-to-end. SLA hardening.

Gate Milestones:

- Feedback loop reducing urgency model error by > 15%
- Household improvement index in funder reports
- External API with 2 third-party integrators
- DPDP compliance audit passed
- Right-to-erasure tested and documented
- System SLA 99.5% uptime over 30-day window

■ Offline-first is not optional

The field app and volunteer app must work without any connectivity. WatermelonDB (SQLite-backed) stores all data locally and syncs when connection is restored. This is not a fallback — it is the primary operating mode for 40% of field deployments in rural areas. Any architecture that assumes connectivity for core operations will fail in production.

■ Human-in-the-loop is a design principle, not a fallback

Every algorithmic output is a ranked suggestion, not a decision. The coordinator interface must make it faster to accept a good suggestion than to override it, but override must always be one tap away. Trust is earned through demonstrated accuracy over time. Full automation is unlocked gradually in Phase 4+. Never ship a system that removes coordinator agency — field realities the algorithm cannot see are always present.

■ 'Invisible NGO' integration cannot require technical sophistication

Most local social groups cannot call a REST API. Integration must be achievable via: WhatsApp bot, Google Sheets plugin (Apps Script reading to CSV), email attachment parsing, or a web form. The ingestion gateway must accept all of these with identical fidelity. Failing to accommodate non-technical ingestion pathways locks out the majority of community organizations.

■ The household registry must model informal identity

Households have no formal addresses. Family composition changes. The same household may have different 'head of household' names with different NGOs. The `landmark_tags` array and `location_description` text field are not fallbacks for missing GPS — they are first-class data because informal location identifiers carry information that coordinates alone cannot. Never overwrite a human-provided description with a geocoder's normalized output.

■ Consent scope must be enforced at the data layer, not just the API layer

Applying consent checks only in API controllers means a compromised or miscoded controller bypasses all privacy protections. Consent scope must be enforced via PostgreSQL column-level security and row-level security policies. A query from a component that lacks the required consent scope must fail at the database level, not rely on application-layer filtering.

■ The language model corpus is mission-critical infrastructure

The NLP models are only as good as the annotated training data. You cannot use generic humanitarian datasets — they will not capture the informal Indian language patterns used in field reports. Budget 6–8 weeks for domain expert annotation before model training begins. This is not a Phase 2 detail — it is a Phase 2 blocker. The annotation effort should begin in Phase 0.

■ Redis distributed lock prevents volunteer double-dispatch

When two coordinators simultaneously approve a task for the same volunteer, without a distributed lock the system sends two dispatch notifications. The task orchestration service acquires a Redis lock on `volunteer_id:date` before sending any dispatch. Lock TTL = 30 seconds. Any concurrent dispatch to the same volunteer within the lock window is rejected and rerouted to the next candidate.

■ Kafka topic schema must be versioned from day one

Kafka events are consumed by multiple services. A schema change in a producing service that breaks a consuming service causes a production incident. Confluent Schema Registry (or AWS Glue Schema Registry) must be adopted in Phase 0. All events use Avro schemas with backward-compatible evolution rules. Never publish unschematized JSON to production Kafka topics.

Service Level Objectives

SLO	Target	Measurement Method
API p99 latency (read)	< 300ms	Measured at load balancer
API p99 latency (write)	< 500ms	Includes DB write + Kafka publish
Matching engine response	< 2 seconds	From task creation to top-3 candidates returned
Ingestion lag	< 30 seconds	From WhatsApp message received to NeedRecord created
OCR pipeline throughput	< 3 minutes	Per paper survey photo, end to end
Dashboard live update	< 2 seconds	WebSocket event propagation from DB write
System uptime	99.5% monthly	Excluding scheduled maintenance windows
Data backup RPO	< 1 hour	RDS automated snapshots + WAL shipping
Recovery time (RTO)	< 4 hours	From declared incident to service restoration
Right-to-erasure execution	< 30 days	Per DPDP Act 2023 requirement

Observability Stack

Metrics	Prometheus scrapes all FastAPI and Node.js services. Custom metrics: kafka_consumer_lag_per_topic, matching_engine_p99_ms, ocr_confidence_histogram, urgency_score_distribution. Grafana dashboards per layer.
Distributed Tracing	Jaeger with OpenTelemetry instrumentation. Every ingestion event carries a trace_id from gateway through OCR → NLP → household match → NeedRecord creation. Coordinator can view the full trace for any need record.
Log Aggregation	ELK stack (Elasticsearch + Logstash + Kibana). Structured JSON logs from all services. Log correlation via trace_id. PII fields are masked before logging — log shipping never touches the PII vault.
Alerting	PagerDuty integration via Alertmanager. P1 alerts (system down, data loss risk): < 5-minute response. P2 alerts (SLO breach, ingestion lag > 2 min): < 30-minute response. P3 alerts (elevated error rates): next business day.
Capacity Planning	Weekly automated report: Kafka partition utilization, PostgreSQL table sizes, Redis memory, S3 bucket growth. Triggers capacity review when any resource hits 70% of provisioned limit.

Volunteer Onboarding & Retention Operations

The system is only as valuable as the quality and quantity of its volunteer pool. Operational processes must be designed alongside the software:

- Onboarding: Digital form + org coordinator verification + skills evidence upload. Target < 10-minute end-to-end registration for a volunteer with the app.
- Retention: Monthly impact summary push notification to each volunteer showing households they helped, hours contributed, and neighborhood impact. Gamification is deliberately avoided — the focus is genuine impact visibility.

- Burnout monitoring: Coordinator receives weekly digest of volunteers approaching burnout threshold ($B > 0.7$).
Proactive check-in workflow built into coordinator portal.
- Skill verification: Automated reminder to re-verify credentials that expire (medical licenses, first aid certifications).
Expired credentials downgrade verified skill to unverified in capability vector.

India's Digital Personal Data Protection Act 2023 imposes specific obligations that must be built into the system architecture — not bolted on as a compliance layer. Each requirement below maps to a concrete implemented system behavior.

DPDP: Lawful Purpose

Every data collection must have a stated, specific purpose. In NEXUS, `household_consent.scope.purposes[]` is mandatory and validated against an allowlist of purpose codes: `needs_assessment`, `resource_dispatch`, `impact_reporting`, `cross_org_coordination`, `anonymized_analytics`. Data collected for one purpose cannot be used for another — enforced by the API layer checking consent scope before returning any field.

DPDP: Data Minimization

The field app's data collection forms are dynamically generated based on the minimum fields required for the current purpose. A field worker assessing food needs does not see mental health history fields. Enforced at API level via purpose-scoped field masks applied to every household read operation.

DPDP: Right to Access

A household primary contact can request a full copy of all data held about them. The household history ledger generates this report automatically. PII vault decrypts only the requesting household's data. Report delivered in the language of original consent.

DPDP: Right to Erasure

Opt-out sets `opt_out=true` in consent record. Triggers cascade: all PII vault entries marked for deletion (executed within 30 days). History ledger entries have personal identifiers replaced with anonymized tokens. The anonymized ledger record is retained for organizational accountability. `Household_id` and aggregated statistics are retained; the person behind them is erased.

DPDP: Consent for Minors

Any `household_member` with `age_bracket` in `{infant, child}` requires parental/guardian consent. The consent record must reference the consenting adult `member_id`. System enforces that child-specific data fields cannot be populated without a valid guardian consent record in scope.

DPDP: Data Localization

All primary data stores (RDS, S3, MSK) provisioned in AWS `ap-south-1` (Mumbai). No personal data is processed outside Indian territory. Google Cloud Vision and Speech-to-Text API calls send image/audio data to Google's infrastructure — field photographs are preprocessed locally (OCR preprocessing via OpenCV on-device) before any cloud API call, minimizing data exposure.

DPDP: Grievance Mechanism

Inbound SMS/WhatsApp keyword 'COMPLAINT' creates a grievance record routed to the organization's data protection officer (stored in `tenants.data_protection_officer_ref`). This channel remains operational even for opted-out households. 30-day response SLA. Grievance log is a separate append-only table accessible only to DPO role.

DPDP: Consent Renewal

All consent records have an `expires_at` field. System sends renewal reminder 14 days before expiry via the household's preferred contact channel. If consent expires without renewal, data access automatically downscopes to `anonymized_analytics` only until renewed. Coordinator is notified of all expired consent records in their service area.

PII Vault Architecture

PII (names, phone numbers, photos) is never stored inline in any operational table. All PII lives in a dedicated encrypted store with the following properties:

- Encryption: AES-256-GCM with AWS KMS managed keys. Key rotation every 90 days. Each tenant has a separate KMS key.
 - Access: Only authenticated API calls from services with the PII_READ IAM policy can decrypt. Every access is logged with: service_id, user_id, household_id, field_accessed, timestamp, purpose_code.
 - Separation: The PII vault database has no network route to the public internet. Access only via VPC private endpoint. Cannot be queried from analytics or Elasticsearch pipelines.
 - Anonymization: When household opts out, PII vault entries are cryptographically shredded (KMS key deleted for that household's data envelope). The pii_ref UUID in household_members becomes a dead pointer. This is true erasure, not soft deletion.
-

NEXUS — Needs Exchange & Unified Coordination System

This document represents the complete system architecture for NEXUS Version 2.0. It incorporates the full Household & Beneficiary Registry Module as a foundational layer, resolving the six identified failure domains of community coordination at scale. All architectural decisions are justified for the Indian NGO operational context, DPDP Act 2023 compliance, and the constraints of rural connectivity and informal community structures.

Document prepared: April 2026 · Version 2.0 · Confidential